

Data Structures & Algorithms

Lecture 2 – Asymptotic Analysis & Arrays

Dr. Yatish Bathla, PhD. (Information Technology), Obuda University, Budapest

Learning outcomes

Define Big-O, Big-Omega and Big-Theta and use them correctly

Simplify a running-time function to its dominant term

Analyse loops: single, nested and halving

Explain how arrays are stored and why access is constant time

Compare linear and binary search, and state the cost of array operations

Why asymptotic analysis?

Exact step counts depend on hardware and language

Small inputs hide differences; large inputs expose them

We compare algorithms by how running time grows with n

Ignore constant factors and lower-order terms

Result: a machine-independent way to compare algorithms

Big-O: an upper bound

f(n) <= c * g(n)

The definition

For all n >= n0, with constants c > 0 and n0 >= 1, we write f(n) is O(g(n)).

Example: 8n + 5 is O(n) because 8n + 5 <= 9n for every n >= 5 (c = 9, n0 = 5).

Big-O, Big-Omega, Big-Theta

O

Big-O

Upper bound: f grows no faster than g. 'At most'.

Omega

Big-Omega

Lower bound: f grows at least as fast as g. 'At least'.

Theta

Big-Theta

Tight bound: both O and Omega. 'Exactly this rate'.

Theta(n) means c1 * n <= f(n) <= c2 * n for large n.

Big-O proofs: finding c and n0

f(n)	Claim	Witnesses
3n + 2	O(n)	c = 4, n0 = 2
2n + 10	O(n)	c = 3, n0 = 10
5 n squared + 3n	O(n squared)	c = 8, n0 = 1
7	O(1)	c = 7, n0 = 1

Rules for simplifying

Drop constant factors: 5n is Theta(n)

Drop lower-order terms: 3n squared + 10n + 7 is Theta(n squared)

Sequential steps add: O(n) + O(n squared) = O(n squared)

Nested steps multiply: n iterations of an O(n) body = O(n squared)

Use the tightest bound you can justify

Reading loops: common patterns

Code pattern	Time
Single statement	O(1)
for i in 1..n	O(n)
for i in 1..n: for j in 1..n	O(n squared)
i = i * 2 until i >= n (halving/doubling)	O(log n)
for i in 1..n with a halving loop inside	O(n log n)

Why growth rate matters in practice

Algorithm class	Steps for n = 1,000,000	Time at 10^9 steps/sec
O(log n)	about 20	20 nanoseconds
O(n)	1,000,000	1 millisecond
O(n log n)	about 20,000,000	20 milliseconds
O(n squared)	1,000,000,000,000	about 17 minutes

Worked example: a halving loop

```
int count = 0;
for (int i = 1; i < n; i *= 2) {
    count++;
}
// n = 16: i = 1, 2, 4, 8
// count = 4
```

Analysis

i doubles each pass

After k passes, i = 2 to the power k

Loop stops when 2^k >= n

So k is about log2(n): O(log n)

Worked example: nested loops

```
int count = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        count++;
    }
}
```

Analysis

Inner body runs n times per outer pass

Outer loop runs n times

Total: n * n = n squared

Running time is Theta(n squared)

Best, average and worst case

Linear search example

Best case: key at index 0, 1 comparison

Worst case: key last or absent, n comparisons

Average case: about n/2 comparisons

Why it matters

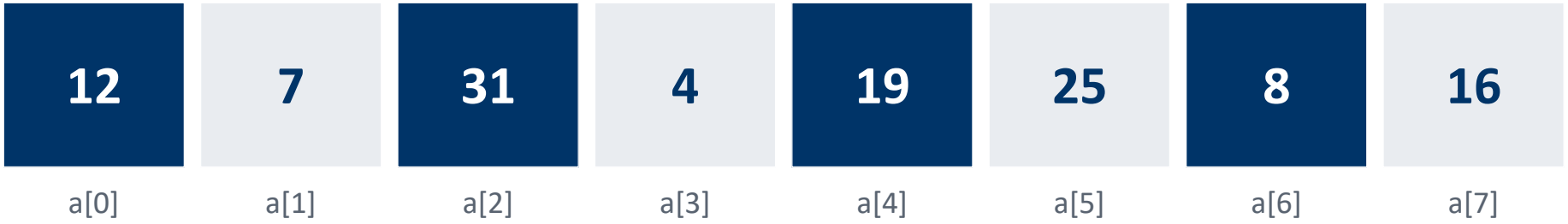
Worst case gives a guarantee

Best case is rarely typical

Average case needs assumptions about the input

All three are still Theta(n) or less in growth

Arrays: contiguous memory



Elements sit side by side, each the same size.

Address of a[i] = base address + i * element size

So reading or writing a[i] is one step: O(1) for any i.

The size is fixed when the array is created (int[] a = new int[8];).

Arrays in Java: the essentials

```
int[] a = new int[5];
int[] b = {4, 8, 15, 16, 23};
a[0] = 42;
int n = b.length;
for (int v : b) {
    System.out.println(v);
}
Arrays.sort(b);
int[] c = Arrays.copyOf(b, 10);
```

Costs

a[0] = 42 is O(1)

Printing all items is O(n)

Arrays.sort is O(n log n)

copyOf creates a new array: O(n)

Two-dimensional arrays

Structure

Array of arrays: int[][] grid = new int[3][4]

Access grid[r][c] is O(1)

Stored row by row

Cost

Visiting every cell: O(rows x columns)

Square n x n grid: O(n squared)

Used for boards, images, matrices

Array operations and their cost

Operation	Unordered array	Ordered array
Access by index	O(1)	O(1)
Search for a value	O(n) linear	O(log n) binary
Insert at end (space free)	O(1)	O(n) to keep order
Insert or delete in the middle	O(n) shifting	O(n) shifting

Binary search trace

Step	lo	hi	mid	a[mid]	Decision
1	0	6	3	15	15 < 27, go right
2	4	6	5	27	found at index 5

Each step halves the remaining range, so at most about log2(n) + 1 steps.

Insertion needs shifting

```
// insert x at index pos
// precondition: size < a.length
for (int i = size; i > pos; i--) {
    a[i] = a[i - 1];
}
a[pos] = x;
size++;
```

Analysis

Shift elements right to make a gap

Worst case: pos = 0, size shifts

Best case: pos = size, no shifts

Cost: O(n) in the worst case

Linear vs binary search

1,000,000

Items in sorted array

Binary search needs the data ordered.

1,000,000

Linear worst case

Up to n comparisons.

20

Binary worst case

About log2(n) comparisons.

Halving the search range each step turns n into log n.

Strengths and limits of arrays

Strengths

O(1) access by index

Compact, cache-friendly storage

Simple to use and reason about

Binary search when ordered

Limitations

Fixed size once created

Insert and delete in the middle cost O(n)

Ordered arrays make insertion slow

Next steps: linked lists and trees

Practice: classify these

Print every element of an array of n items

Print every pair of elements

Read a[n/2] from an array

Search a sorted array of n items by repeated halving

Answers: O(n), O(n squared), O(1), O(log n)

Summary and exit ticket

Big-O upper bound, Big-Omega lower bound, Big-Theta tight bound

Drop constants and lower-order terms; nested loops multiply

Array access is O(1); search O(n) or O(log n) if ordered

Insert and delete in the middle cost O(n) because of shifting

Exit ticket: why is binary search O(log n)? Lab next: implement and measure